

Lens IQ

Full Project Evaluation

AI-Powered Financing Decision Engine for Car Dealerships

Generated: 21 June 2026

Project: car1-fintech / lens-iq

Monorepo | Turborepo | Fastify 5 | React 19 | PostgreSQL

Table of Contents

1. Executive Summary
2. Project Overview
3. Technology Stack
4. Architecture & System Design
5. Directory Structure
6. Database Schema (10 Models)
7. API Endpoints Reference
8. Core Engine: Evaluation Pipeline
9. Scoring & Risk Analysis
10. Offer Generation & Comparison
11. Admin Dashboard (Frontend)
12. Widget (CRM Embed)
13. Authentication & Security
14. Deployment & Infrastructure
15. Code Quality & Patterns
16. Evaluation & Recommendations
17. Strengths & Weaknesses
18. Conclusion

1. Executive Summary

Lens IQ is a multi-tenant, AI-powered financing decision engine designed specifically for car dealerships. It enables dealerships to instantly evaluate customer financing applications against multiple bank programs, generate optimized offers, and increase approval rates — all without replacing their existing CRM. The system is built as a monorepo using Turborepo with npm workspaces, comprising three applications (API backend, Admin Dashboard, Embeddable Widget) and six shared packages.

Key capabilities include a configurable rule engine, DTI-based eligibility checks, risk scoring, loan pricing with multiple calculation methods (reducing, flat, murabaha), offer ranking, and optimization suggestions. The system supports Arabic language for public-facing endpoints, RBAC (Admin/Manager/Sales Agent), and is deployment-ready with Docker and Railway configuration.

2. Project Overview

Name: Lens IQ (@lens-iq)

Version: 1.0.0

Package Manager: npm 11.9.0 (workspaces)

Monorepo Tool: Turborepo 2.5

License: ISC

Target Users: Car dealerships (multi-tenant)

Primary Language: TypeScript (ESM, ES2022)

Business Goals

- Increase financing approval rates for dealerships
- Reduce qualification time from days to seconds
- Optimize financing offers across multiple banks/programs
- Provide embeddable widget for existing CRM systems
- Support both conventional and Islamic financing

3. Technology Stack

Runtime: Node.js (TypeScript ESM)
API Framework: Fastify 5 (v5.8.5)
Frontend (Admin): React 19 + TypeScript + Vite 8
UI Library: MUI 9 + Tailwind CSS 4
State Management: Zustand 5 + TanStack React Query 5
Widget: React 19 + Vite 8 (lightweight iframe)
Database: PostgreSQL (via pg + Prisma 6 ORM)
Validation: Zod 4
Auth: JWT + bcrypt + Refresh Tokens
Forms: react-hook-form
Animation: Framer Motion 12
Notifications: sonner + lucide-react
Deployment: Docker + Railway
Linting: ESLint + Prettier

4. Architecture & System Design

The system follows a layered monorepo architecture with clear separation of concerns:

System Flow

1. CRM Integration: Dealership CRM integrates via embeddable iframe Widget or REST API
2. API Gateway: Fastify server receives applications and runs the Evaluation Engine
3. Engine Pipeline: Policy Rules !' Eligibility (DTI) !' Risk Scoring !' Decision !' Offer Generation
4. Offer Ranking: All generated offers are compared and ranked by approval probability
5. Persistence: Offers are stored in PostgreSQL with full audit trail
6. Admin Dashboard: Dealership staff manage banks, programs, rules, customers, vehicles, users

Multi-Tenancy

Every table is scoped by `tenant_id` with CASCADE deletes. Each tenant (dealership) gets an API key at registration. All queries filter by `tenant_id`, ensuring complete data isolation.

RBAC

Three roles: ADMIN (full access), MANAGER (limited admin), SALES_AGENT (evaluation only). Middleware checks role on protected routes. Managers can have subordinates (Sales Agents).

5. Directory Structure

The project is organized as a Turborepo monorepo with npm workspaces. Below is the logical structure:

lens-iq/

```

package.json      # Root config, scripts
turbo.json        # Turborepo orchestration
tsconfig.json     # Shared TS config (ES2022)
Dockerfile        # Production build
railway.json      # Railway deployment config
apps/
  api/            # Fastify 5 REST API
    src/
      server.ts   # Entry point, plugins, routes
      auth/       # JWT auth, RBAC, cookies
      admin/      # CRUD: banks, programs, rules, customers, vehicles, applications, users
      engine/     # Core financing engine
        evaluation/ # Pipeline: policy, eligibility
        scoring/  # Risk scoring, DTI, affordability
        offers/   # Generator, comparison, ranking
        pricing/  # Loan calculator (reducing/flat/murabaha)
        rules/    # Rule evaluator + operators
        builders/ # Result builder
        types/    # Engine types
      routes/     # Route handlers (evaluate, optimize, public, dashboard)
      services/   # Business logic services
      db/         # pg Pool connection
      mappers/    # Data mappers
      cli/        # CLI scripts
  admin-dashboard/ # React 19 SPA
    src/
      features/   # Feature modules (applications, banks, programs, etc.)
      components/ # Reusable UI (layout, data-display, feedback)
      router/     # React Router config
      store/      # Zustand store
      api/        # Axios client
      app/        # Providers (React Query)
  widget/        # Embeddable iframe widget
packages/
  db/            # Prisma schema, migrations, seed
  types/         # Shared TypeScript interfaces
  utils/         # Utility functions (clamp, cn, format)
  config/        # Runtime config object
  ui/           # UI library (placeholder)
  shared-types/  # Cross-app types
  docs/         # Documentation (API, audit, deployment, Arabic docs)

```


6. Database Schema

The database uses PostgreSQL with Prisma ORM. There are 10 models with 6 enums:

Tenant

Dealership account with unique API key

Fields: id, name, email, password_hash, api_key, role, active, max_users

User

Sub-users with RBAC under a tenant

Fields: id, tenant_id, manager_id, name, email, password_hash, role, active

RefreshToken

JWT refresh token storage

Fields: id, tenant_id, user_id, token_hash, expires_at

Bank

Financial institutions per dealership

Fields: id, tenant_id, name, code, logo_url, active

Program

Financing programs with rates, rules, eligibility

Fields: id, tenant_id, bank_id, name, financing_type, calculation_method, min_salary, max_customer_age, interest_rate, min/max_months, min_down_payment_percent, etc.

Rule

Configurable rules per program

Fields: id, tenant_id, program_id, field, operator, value, action

Customer

Customer demographics & financial data

Fields: id, tenant_id, name, national_id, phone, birth_date, salary, job_type, current_liabilities, additional_income, etc.

Vehicle

Vehicle inventory

Fields: id, tenant_id, brand, model, manufacturing_year, condition, price, category

Application

Financing applications linking customers to vehicles

Fields: id, tenant_id, customer_id, vehicle_id, requested_down_payment, requested_months, payment_method, status

Offer

Generated financing offers with full pricing & risk data

Fields: id, tenant_id, application_id, program_id, bank_id, status, installment, total_payment, finance_amount, down_payment, interest_rate, months, dti, risk_score, risk_level, affordability_score, reasons (JSON)

Enums

ApplicationStatus (PENDING, APPROVED, REJECTED, CANCELLED, UNDER_REVIEW), JobType (private, government, corporate, freelancer, retired), VehicleCondition (new, used), VehicleCategory

(sedan, suv, truck, van, microbus), FinancingType (conventional, islamic), CalculationMethod (reducing, flat, murabaha), AllowedConditions (new, used, both), PaymentMethod (salary_transfer, bank_account, cash_proof), Role (ADMIN, MANAGER, SALES_AGENT)

7. API Endpoints Reference

All endpoints return `{ success: boolean, data?: T, message?: string }`.

Endpoint	Description	Auth
POST /auth/register (tenant)	Register dealership	
POST /auth/login	Login, receive JWT + refresh cookie	
POST /auth/refresh	Refresh JWT token	
POST /auth/logout	Logout, clear cookie	
GET /me	Current user/tenant info	
PATCH /auth/profile	Update tenant name	
PATCH /auth/password	Change password	
POST /auth/regenerate-key	Regenerate API key	JWT
POST /evaluate	Evaluate application against all programs	
POST /optimize	Get optimization suggestions	
GET/POST /admin/banks	List/Create banks	
PATCH/DELETE /admin/banks/:id	Update/Delete bank	JWT
GET/POST /admin/programs	List/Create programs	
GET/POST /admin/rules/:programId	List/Create rules per program	JWT
GET/POST /admin/customers	List/Create customers	
GET /admin/customers/:id	Get customer by ID	JWT
GET/POST /admin/vehicles	List/Create vehicles	
GET /admin/vehicles/:id	Get vehicle by ID	
GET/POST /admin/applications	List/Create applications	
PATCH /admin/applications/:id/status	Update status	JWT
GET /admin/users	List users (ADMIN/MANAGER)	
POST /admin/users	Create SUB-user (ADMIN)	
GET /dashboard	Dashboard Statistics	
GET /public/vehicles/:code	Public vehicle listing	API Public
POST /public/apply (Arabic support)	Public application	
GET /health	Health check (db connectivity)	

8. Core Engine: Evaluation Pipeline

The financing decision engine is the heart of Lens IQ. It follows a deterministic, composable pipeline pattern:

Step 1: Load Rules

Rules are loaded from the database for the specific program being evaluated. Each rule has: field (e.g., salary, age, car_age), operator (=, !=, >, >=, <, <=), value, and action.

Step 2: Policy Engine (`policyEngine.ts`)

The policy engine evaluates all rules for the program. If any rule fails (action = REJECT), the application is immediately rejected with the rule reason. This is a gating mechanism.

Step 3: Eligibility (`eligibility.ts`)

Calculates the Debt-to-Income (DTI) ratio: $\text{DTI} = (\text{installment} + \text{monthly_liabilities}) / \text{salary} * 100$. If DTI exceeds 65%, the application is rejected. The base DTI is saved for downstream use.

Step 4: Risk Scoring (`riskScore.ts`)

Evaluates three factors: age (higher risk if > 60), salary (higher risk if < 5000), and DTI. Produces a numeric score (0-100) and level (LOW: 0-30, MEDIUM: 31-60, HIGH: 61+).

Step 5: Decision

Based on risk level:

- LOW != APPROVE (no modifiers)
- MEDIUM != CONDITIONAL (interest +10%)
- HIGH != REJECT (but interest +25%, months -12 are stored for analysis)

Step 6: Offer Generation (`offerGenerator.ts`)

For APPROVED/CONDITIONAL decisions: calculates final interest rate (base + modifier), final months (base + modifier), minimum down payment, loan amount, and generates full pricing via the loan calculator. Re-runs scoring with actual installment, then computes approval probability.

Step 7: Offer Comparison & Ranking

All offers across all active programs are compared. Approved offers are ranked by approval probability (descending) using `rankOffers()`. The best offer is returned as `bestOffer`.

9. Scoring & Risk Analysis

DTI Calculation (`dti.ts`)

$\text{DTI} = ((\text{installment} + \text{current_liabilities}) / \text{salary}) * 100$. Max allowed DTI is 65%. This is the primary financial health metric.

Risk Score (`riskScore.ts`)

The risk function evaluates:

- Age risk: age > 60 adds 40 points
- Salary risk: salary < 5000 adds 30 points
- DTI risk: DTI * 0.6 (up to 60 points)
- Total: 0-100 (LOW < 30, MEDIUM 31-60, HIGH > 60)

Affordability Score (`affordability.ts`)

Linear interpolation: DTI 0% != 100 score, DTI 65% != 0 score. Clamped to [0, 100]. Higher is better.

Approval Probability

For APPROVED: $\text{`affordability`} * 0.5 + (100 - \text{riskScore}) * 0.3 + (100 - \text{DTI}) * 0.2$. For REJECTED: $\text{`50 - (DTI * 0.3 + riskScore * 0.4)}`$. Clamped to [5, 99].

10. Offer Generation & Pricing

Loan Calculator (`loanCalculator.ts`)

Three calculation methods with modifier support:

- Reducing Balance: $M = P * [r(1+r)^n] / [(1+r)^n - 1]$ (standard amortization)
- Flat Rate: $\text{Interest} = P * r * (n/12)$, $\text{Installment} = (P + \text{Interest}) / n$
- Murabaha (Islamic): $\text{Profit} = P * r * (n/12)$, same formula as flat but labeled as profit

Rate Convention

Annual rate is stored as percentage (e.g., 14.5 = 14.5% per year). Interest modifiers are percentage points (e.g., +10 = +10%). Monthly rate = $\text{annualRate} / 12 / 100$.

Offer Comparison (`compareOffers.ts`)

Iterates all active programs for a tenant, evaluates each against the application input, generates an offer, and returns the complete offer list. Clears old offers before persisting new ones.

11. Admin Dashboard (Frontend)

The admin dashboard is a React 19 SPA built with Vite 8, MUI 9, and Tailwind CSS 4. It uses TanStack React Query 5 for server state, Zustand 5 for client state (auth), and Framer Motion 12 for animations.

Feature Modules

- Applications: List, create, evaluate, view offers
- Banks: CRUD management with status toggle
- Customers: CRUD with financial data
- Vehicles: CRUD with brand/model/condition
- Programs: CRUD with full financing parameters
- Rules: Configurable rules per program
- Users: Sub-user management with RBAC
- Evaluate: Full evaluation UI with offer comparison, risk gauge, DTI meter
- Dashboard: Statistics and metrics
- Apply: Public application form
- Profile: Tenant profile management
- Auth: Login page

Key Components

- DataTable: Reusable sortable table with search
- DecisionCard: Visual decision display (APPROVED/CONDITIONAL/REJECTED)
- OfferComparison: Side-by-side offer comparison table
- RiskScoreGauge: Radial gauge for risk visualization
- DtiMeter: Horizontal bar for DTI display
- StatusBadge: Color-coded status badges
- Recommendations: Optimization suggestions with impact indicators

12. Widget (CRM Embed)

The widget is a lightweight React 19 application built with Vite 8. It is designed to be embedded in dealership CRM systems via an iframe. The widget communicates with the API via the public endpoints (API-key authenticated) and serves as a minimal application intake form.

The widget is a single-page component (`Widget.tsx`) that:

- Loads available vehicles for the dealership
- Collects customer information and financial data
- Submits the application via POST `/public/apply`
- Displays the evaluation result with offer details

13. Authentication & Security

Authentication Flow

- Registration creates a Tenant with hashed password (bcrypt) and auto-generated API key
- Login validates credentials and returns JWT (access token) + sets HTTP-only refresh cookie
- JWT contains tenantId, userId, and role for RBAC
- Refresh token stored hashed in DB, rotated on each use
- Logout clears the refresh cookie and deletes the token from DB

Security Measures

- Rate limiting: 100 req/min global, 5 req/15min on login
- Security headers: CSP, HSTS, X-Frame-Options DENY, X-Content-Type-Options nosniff
- Cookie secret for signing refresh cookies
- RBAC middleware on all admin routes
- API key authentication for public endpoints
- CORS whitelist for allowed origins
- Request ID tracing (8-char UUID on each request)
- Global error handler (no stack leaks in production)

14. Deployment & Infrastructure

The project is deployment-ready with multiple configurations:

- Dockerfile: Multi-stage production build at root level
- Railway Configuration: `railway.json` at root and per-app level
- Frontend Serving: API automatically serves admin-dashboard static files in production
- SPA Fallback: `setNotFoundHandler` returns `index.html` for non-API GET routes
- Graceful Shutdown: `SIGTERM/SIGINT` handlers close Fastify and DB pool
- Environment Validation: Zod schema validates all required env vars at startup
- NGINX Config: `nginx-default.conf` for admin-dashboard in production

Environment Variables

- `DATABASE_URL` - PostgreSQL connection string
- `JWT_SECRET` - JWT signing secret
- `COOKIE_SECRET` - Cookie signing secret
- `PORT / HOST` - Server binding

- CORS_ORIGINS - Allowed CORS origins
- LOG_LEVEL - Logging verbosity
- FRONTEND_DIST - Custom frontend path

15. Code Quality & Patterns

Monorepo Structure

- npm workspaces with 6 internal packages: @lens/db, @lens/types, @lens/utils, @lens/config, @lens/ui, @lens/shared-types
- Turborepo for build orchestration with caching
- ESM throughout ("type": "module" in all packages)
- Explicit .js extensions in TypeScript imports (ESM convention)

API Architecture

- Controller/Service/Route separation
- Feature-based organization (admin/banks, admin/programs, etc.)
- Consistent response format: { success, data?, message? }
- Dual DB access: Prisma (schema/migrations) + raw pg Pool (runtime)
- Zod validation on route inputs

Frontend Architecture

- Feature-based modularity (each feature has api/, pages/, components/)
- Reusable components: DataTable, Card, Skeleton, ErrorBoundary
- TanStack React Query for server state caching and invalidation
- Zustand store for auth state persistence
- Centralized API client (Axios) with interceptors

Engine Architecture

- Pure functions with no side effects (scoring, pricing)
- Composable pipeline pattern
- Type-safe with shared interfaces
- Deterministic evaluation (same input != same output)

16. Evaluation & Recommendations

Strengths

- Comprehensive multi-tenant architecture with proper data isolation
- Sophisticated rule engine allows dealerships to configure custom financing criteria
- Multiple calculation methods support both conventional and Islamic financing
- Clean pipeline architecture in the evaluation engine (deterministic, testable, composable)
- Excellent API design with consistent response format and comprehensive endpoints
- Full RBAC with three roles and subordinate management
- Production-ready: Docker, Railway, env validation, graceful shutdown, security headers
- Bilingual support (Arabic for public endpoints)
- Rich frontend with modern stack (React 19, MUI 9, Tailwind 4, TanStack Query 5)
- Comprehensive documentation (API reference, audit, deployment, Arabic docs)

Areas for Improvement

- Testing: No test files found in the codebase. Unit tests for the engine (scoring, pricing, rules) and integration tests for API endpoints are critical for a financial system.
- TypeScript strictness: No ``strict: true`` in tsconfig, some ``any`` types used in catch blocks and route handlers.
- Error handling in engine: The loan calculator throws on invalid input; a Result type would be more composable.
- Database access: Mixing Prisma and raw pg queries adds complexity. Consider standardizing on one approach.
- Logging: Fastify's built-in logger is used but no structured logging or monitoring integration.
- Rate limiting: Static config (100 req/min). Consider making this configurable per-tenant.
- API key management: Keys are generated with a simple random approach; consider a more robust generation strategy.
- No CI/CD configuration (GitHub Actions, etc.) in the repository.
- Some frontend type definitions could be shared more consistently with the API types.
- The widget is very minimal; could benefit from more comprehensive UI and error states.

17. Overall Assessment

Lens IQ is a well-architected, production-grade financing decision engine. The code is clean, the architecture follows modern best practices (monorepo, ESM, feature-based modules), and the core engine is elegantly designed as a deterministic pipeline of pure functions.

The system demonstrates strong understanding of the car financing domain, supporting multiple calculation methods, configurable rules, risk scoring, and offer optimization. The multi-tenant design with RBAC makes it suitable for real-world dealership deployments.

The most significant gap is the complete lack of tests, which is a critical concern for any financial decision-making system. Adding unit tests for the engine and integration tests for the API should be the top priority. Beyond that, the project is well-positioned for production use.

18. Conclusion

Lens IQ is a comprehensive, well-engineered financing decision platform. It combines a sophisticated rule-based evaluation engine with a modern React dashboard and embeddable widget, all built on a solid monorepo architecture.

The project successfully addresses the core business need: helping car dealerships quickly evaluate financing applications across multiple bank programs, optimize offers, and increase approval rates. The multi-tenant design, RBAC, bilingual support, and deployment-ready infrastructure make it a complete, production-ready solution.

With the addition of comprehensive testing, the project would be well-positioned for real-world deployment and scaling.

19. Audit & Fixes Log (2026-06-21)

Issues Found & Fixed

During the comprehensive code review on 2026-06-21, the following issues were identified and resolved:

FIX 1: Widget API Endpoint URL Mismatch

File: `apps/widget/src/Widget.tsx:30` — The widget was calling `/api/v1/evaluate` but the API registers the route at `/evaluate` (`apps/api/src/routes/evaluate.ts:9`). This caused a 404 error when the widget tried to fetch evaluation results.

Fix: Changed the fetch URL from `/api/v1/evaluate` to `/evaluate` to match the actual API route.

FIX 2: Sub-User Login Returns Wrong Tenant ID

File: `apps/api/src/auth/auth.service.ts:113-114` — When a sub-user (not the main tenant) logs in, the response's `tenant.id` was incorrectly set to `user.id` instead of `user.tenant_id`. This caused the frontend to use the wrong ID for tenant-scoped operations.

Fix: Changed `tenant.id` from `user.id` to `user.tenant_id`.

FIX 3: ESLint Errors — no-explicit-any

File: `apps/admin-dashboard/src/features/users/pages/UsersPage.tsx` — Four ESLint errors where `err` parameters in `onError` callbacks were typed as `any`. This violated the `@typescript-eslint/no-explicit-any` rule.

Fix: Changed all `any` types to `Error`, matching TanStack React Query v5's default error type.

FIX 4: Cookie Secret Warning

File: `apps/api/src/shared/env.ts` — Added a startup warning when `COOKIE_SECRET` is still the default value (default-`cookie-secret-change-me`), matching the existing pattern for `JWT_SECRET` and `REFRESH_TOKEN_SECRET` warnings.

Security Review Summary

The following security measures are confirmed active:

- Rate limiting: 100 req/min global, 5 req/15min on `/auth/login`
- Security headers: CSP, HSTS, X-Frame-Options DENY, X-Content-Type-Options nosniff, X-XSS-Protection
- `httpOnly` cookies for JWT tokens (inaccessible to JavaScript)
- `SameSite=Lax` cookie policy (CSRF protection)
- `secure` flag enabled in production (HTTPS-only cookies)
- CORS whitelist with credential support
- `bcrypt` password hashing (10 salt rounds)
- JWT with configurable expiry (15m access, 7d refresh)
- Refresh token rotation (old token invalidated on refresh)
- API key authentication for public endpoints
- RBAC middleware on all admin routes
- Global error handler (no stack trace leakage)
- Request ID tracing (8-char UUID on each request)
- Input validation with `Zod` on all endpoints

Database Architecture: Prisma + Raw SQL

The project uses a dual database access strategy by design:

- Prisma (packages/db/): Schema definition, migration management, seed data
- Raw pg Pool (apps/api/src/db/db.ts): All runtime database operations

This separation is valid for a monorepo where Prisma handles schema evolution and the API uses a lightweight pg Pool for runtime queries. The @lens/db package exports a PrismaClient singleton that is available but unused at runtime. Consider migrating to Prisma Client for runtime queries in future iterations to benefit from type-safe queries, auto-completion, and reduced raw SQL maintenance.

Build Verification

All fixes verified with successful builds:

- npm run build:api — TypeScript compilation SUCCESS
- npm run build:fe — TypeScript + Vite build SUCCESS
- npm run db:generate — Prisma client generation SUCCESS
- npx tsc --noEmit (apps/api) — Type check SUCCESS

Project Metrics Summary

- Total Applications:** 3 (api, admin-dashboard, widget)
- Shared Packages:** 6 (@lens/db, @lens/types, @lens/utils, @lens/config, @lens/ui, @lens/shared-types)
- Database Models:** 10 (Tenant, User, RefreshToken, Bank, Program, Rule, Customer, Vehicle, Application, Offer)
- API Endpoints:** 26+ (auth, admin CRUD, evaluation, optimization, public)
- Engine Pipeline Steps:** 6 (Load Rules !' Policy !' Eligibility !' Risk !' Decision !' Offer)
- Languages:** TypeScript (99%), SQL, Shell
- Frontend Components:** 30+ (pages, components, layouts, data display)
- Documentation Files:** 8 (Arabic + English)

--- End of Report ---